

Tutorium 12

Sortieralgorithmen

Grundlagen der Programmierung

15. Juni 2026

Quadratische Sortieralgorithmen $\mathcal{O}(n^2)$

Log-Lineare Sortieralgorithmen $\mathcal{O}(n \log n)$

Praxis

Selection Sort & Bubble Sort

Beide Algorithmen haben eine worst-case Laufzeit von $\mathcal{O}(n^2)$.

Selection Sort

1. Finde das kleinste Element.
2. Entferne es aus der Liste.
3. Füge es an die sortierte Teilliste an.
4. Wiederholen.

Bubble Sort

1. Gehe durch die Liste.
2. Vergleiche benachbarte Elemente.
3. Tausche sie, falls sie falsch herum stehen.
4. Wiederholen bis sortiert.

Insertion Sort

Ebenfalls $\mathcal{O}(n^2)$. Sehr intuitiv (ähnlich wie man Spielkarten sortiert).

Ablauf

1. Nimm das nächste Element aus der unsortierten Liste.
2. Finde die **korrekte Stelle** in der bereits sortierten Folge S .
3. Füge es in S ein.
4. Wiederholen.

Beispiel ($S \mid F$):

() | (21, 5, 3)

(21) | (5, 3)

(5, 21) | (3)

(3, 5, 21) | ()

Ein klassischer Divide-and-Conquer-Algorithmus mit garantierter Laufzeit $\mathcal{O}(n \log n)$.

Ablauf

1. **Teilen:** Teile die Liste in der Mitte in zwei Hälften F_1, F_2 .
2. **Rekursion:** Führe Merge Sort auf F_1 und F_2 aus.
3. **Mergen:** Füge die sortierten Hälften wieder (in linearer Zeit) zusammen.

Beispiel-Baum: $(21, 5, 3, 17) \rightarrow (21, 5) \text{ und } (3, 17) \rightarrow (21), (5), (3), (17) \rightarrow \dots$

Sehr schnell in der Praxis, aber worst-case $\mathcal{O}(n^2)$. Average-case $\mathcal{O}(n \log n)$.

Ablauf

1. **Pivot wählen:** Wähle ein Pivotelement.
2. **Partitionieren:** Teile in F_1 (kleiner als Pivot) und F_2 (größer/gleich).
3. **Rekursion:** Sortiere F_1 und F_2 .
4. **Zusammensetzen:** $F_1 + +[Pivot] + +F_2$

Live Demo



Altklausuraufgaben zu Sortieralgorithmen

Wir spielen die Algorithmen live an Beispielen durch!